

20.6 Robust fitting as a diagnostic

Least squares hides bad cases by accommodating them. A fit that refuses to accommodate them leaves them with large residuals, visible without any deletion. That is the diagnostic use of robust regression, and this section develops only as much theory as that use needs: does a fit the bad cases cannot capture tell a different story, and which cases account for the difference?

20.6.1 M-estimators

Least squares minimizes $\sum_i \rho(y_i - \mathbf{x}_{(i)}^\top \mathbf{b})$ with $\rho(u) = u^2/2$, which lets one large residual outweigh many small ones. An M-estimator (Huber 1964) uses a ρ that grows more slowly.

Definition 20.6.1. M-estimator of regression

Let $\rho : \mathbb{R} \rightarrow [0, \infty)$ be even, nondecreasing on $[0, \infty)$, differentiable, with $\rho(0) = 0$, and put $\psi = \rho'$ and $w(u) = \psi(u)/u$ (with $w(0) = \lim_{u \rightarrow 0} \psi(u)/u$). Given a scale $\hat{\sigma} > 0$, an **M-estimate** $\hat{\mathbf{b}}_M$ minimizes

$$Q(\mathbf{b}) = \sum_{i=1}^n \rho\left(\frac{y_i - \mathbf{x}_{(i)}^\top \mathbf{b}}{\hat{\sigma}}\right).$$

Two choices of ρ are standard.

- (a) **Huber**: $\rho(u) = u^2/2$ for $|u| \leq k$ and $k|u| - k^2/2$ beyond, so $\psi(u) = \max(-k, \min(k, u))$ and $w(u) = \min(1, k/|u|)$.
- (b) **Bisquare** (Tukey's biweight): $\rho(u) = \frac{c^2}{6} [1 - (1 - u^2/c^2)^3]$ for $|u| \leq c$ and $c^2/6$ beyond, so $\psi(u) = u(1 - u^2/c^2)^2$ and $w(u) = (1 - u^2/c^2)^2$ for $|u| < c$, and $\psi(u) = w(u) = 0$ beyond.

Setting the gradient of Q to zero gives the estimating equations

$$\sum_{i=1}^n \psi\left(\frac{y_i - \mathbf{x}_{(i)}^\top \mathbf{b}}{\hat{\sigma}}\right) \mathbf{x}_{(i)} = \mathbf{0} \iff \mathbf{X}^\top \mathbf{W}(\mathbf{b})(\mathbf{y} - \mathbf{X}\mathbf{b}) = \mathbf{0}, \quad (20.6.1)$$

where $\mathbf{W}(\mathbf{b})$ is diagonal with entries $w((y_i - \mathbf{x}_{(i)}^\top \mathbf{b})/\hat{\sigma})$: the normal equations of weighted least squares (Section 6.9) with weights that depend on the fit. A residual beyond $k\hat{\sigma}$ has its Huber influence capped; a residual beyond $c\hat{\sigma}$ gets bisquare weight zero. The Huber ρ is convex; the bisquare ρ is not, and is called **redescending** because ψ falls back to zero.

A standard scale is $\hat{\sigma} = \text{median}_i |y_i - \mathbf{x}_{(i)}^\top \mathbf{b}| / 0.6745$, where $0.6745 = \Phi^{-1}(0.75)$ makes it consistent for σ under centred normal errors; software re-estimates it at each step. The constants $k = 1.345$ and $c = 4.685$ make the location efficiency $(E \psi'(Z))^2 / E \psi(Z)^2$, $Z \sim N(0, 1)$, equal to 0.950 for both (computed by numerical integration in the script). For convex ρ such as Huber's, with p fixed and under smoothness conditions on ψ , that this ratio governs the asymptotic variance of regression M-estimators when $\max_i h_{ii} \rightarrow 0$ is a theorem of Huber (1973) that this book does not prove; redescending estimators such as the bisquare also need a consistent starting value, as in the MM-estimator of Yohai (1987).

20.6.2 Iteratively reweighted least squares

Eq. 20.6.1 suggests **iteratively reweighted least squares** (IRLS), studied for robust regression by Holland and Welsch (1977): from $\mathbf{b}^{(t)}$, compute $w_i = w((y_i - \mathbf{x}_{(i)}^\top \mathbf{b}^{(t)})/\hat{\sigma})$ and let $\mathbf{b}^{(t+1)}$ be the weighted least squares estimate; repeat. With the scale fixed, it never goes uphill.

Proposition 20.6.1. IRLS descends

Suppose $\rho(u) = g(u^2)$, where $g : [0, \infty) \rightarrow [0, \infty)$ is concave, nondecreasing, and differentiable on $(0, \infty)$ with a finite limit $g'(0^+)$. Then $w(u) = 2g'(u^2) \geq 0$. Hold $\hat{\sigma}$ fixed, and suppose $\mathbf{X}^\top \mathbf{W} \mathbf{X}$ is nonsingular at every step. Then every IRLS step satisfies $Q(\mathbf{b}^{(t+1)}) \leq Q(\mathbf{b}^{(t)})$. The Huber and bisquare ρ satisfy the hypotheses.

Proof. Since $\rho(u) = g(u^2)$, $\psi(u) = 2ug'(u^2)$, so $w(u) = 2g'(u^2)$, which is nonnegative because g is nondecreasing. A concave function lies below each of its tangent lines: $g(v) \leq g(v_0) + g'(v_0)(v - v_0)$ for $v \geq 0$, $v_0 > 0$ (and for $v_0 = 0$ with $g'(0^+)$, by continuity). With $v = u^2$ and $v_0 = u_0^2$,

$$\rho(u) \leq \rho(u_0) + \frac{1}{2}w(u_0)(u^2 - u_0^2).$$

Write $u_i(\mathbf{b}) = (y_i - \mathbf{x}_{(i)}^\top \mathbf{b})/\hat{\sigma}$ and $w_i = w(u_i(\mathbf{b}^{(t)}))$. Summing over i ,

$$Q(\mathbf{b}) \leq G(\mathbf{b}) = Q(\mathbf{b}^{(t)}) + \frac{1}{2} \sum_i w_i (u_i(\mathbf{b})^2 - u_i(\mathbf{b}^{(t)})^2) \quad \text{for every } \mathbf{b},$$

with equality at $\mathbf{b} = \mathbf{b}^{(t)}$. Up to terms that do not involve $\mathbf{b}$, $G(\mathbf{b})$ is $\frac{1}{2\hat{\sigma}^2} \sum_i w_i (y_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2$, which the weighted least squares step minimizes. Hence $Q(\mathbf{b}^{(t+1)}) \leq G(\mathbf{b}^{(t+1)}) \leq G(\mathbf{b}^{(t)}) = Q(\mathbf{b}^{(t)})$.

For Huber, $g(v) = v/2$ for $v \leq k^2$ and $k\sqrt{v} - k^2/2$ beyond, so $g'(v) = \frac{1}{2}$ and then $k/(2\sqrt{v})$: continuous at $v = k^2$, nonincreasing, hence g is concave. For the bisquare, $g(v) = \frac{c^2}{6} [1 - (1 - v/c^2)^3]$ for $v \leq c^2$ and constant beyond, so $g'(v) = \frac{1}{2}(1 - v/c^2)^2$, falling to 0 at $v = c^2$, and 0 beyond: again nonincreasing and continuous. ■

This is majorization: Q is replaced by a quadratic that lies above it and touches it at the current point. For Huber's convex ρ , a fixed point of IRLS solves Eq. 20.6.1 and is a global minimizer. For the bisquare, Q can have several local minima, and IRLS finds one near its start.

```
import numpy as np

def huber_weight(u, k=1.345):
    return np.minimum(1.0, k / np.maximum(np.abs(u), 1e-12))

def bisquare_weight(u, c=4.685):
    return np.where(np.abs(u) < c, (1 - (u / c) ** 2) ** 2, 0.0)

def mad_scale(r):
    return np.median(np.abs(r)) / 0.6745          # consistent for sigma at the
                                                normal

def irls(X, y, weight, beta=None, scale=None, tol=1e-10, max_iter=500):
```

```

"""M-estimate by iteratively reweighted least squares.

Starts from least squares unless `beta` is given. If `scale` is None it is re-
estimated
from the current residuals at every step; otherwise it is held fixed."""
if beta is None:
    beta, *_ = np.linalg.lstsq(X, y, rcond=None)
for _ in range(max_iter):
    r = y - X @ beta
    s = mad_scale(r) if scale is None else scale
    w = weight(r / s)
    sw = np.sqrt(w)
    new, *_ = np.linalg.lstsq(X * sw[:, None], y * sw, rcond=None)
    if np.max(np.abs(new - beta)) < tol * (1 + np.max(np.abs(beta))):
        beta = new
        break
    beta = new
r = y - X @ beta
s = mad_scale(r) if scale is None else scale
return beta, weight(r / s), s

```

[Run this code online](#)

20.6.3 Breakdown

How many bad cases can an estimator tolerate? The finite-sample breakdown point of Definition 19.4.1 counts the cases that must be replaced to make the estimate arbitrary, and least squares breaks down at a single case: changing y_i by Δ moves $\hat{\beta}$ by $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_{(i)} \Delta$ (Proposition 19.4.2).

Provided no single row dominates the design, a Huber estimate cannot be broken by changing y_i alone, since ψ is bounded; how much a y -outlier can do depends on the leverages. But one case bad in both $\mathbf{x}$ and y , a **bad leverage point**, breaks it: the fit turns towards the case, whose residual becomes moderate and whose large row then dominates the estimating equations (Exercise (C1)). This is masking again: M-estimates downweight by residual, and a high-leverage case controls its own residual. For regression M-estimates with monotone ψ the breakdown point is $1/n$ (Maronna, Martin and Yohai 2006). Added to the thirty clean cases of Example (Four identical bad records), one case at $x = 50$, $y = 0$ turns the least squares slope from 0.514 to -0.025 , and the Huber slope to -0.026 .

20.6.4 High-breakdown fits

Rousseeuw (1984) proposed estimators that fit a majority of the data and ignore the rest.

Definition 20.6.2. Least trimmed squares

Let q be an integer with $n/2 \leq q \leq n$. For $\mathbf{b} \in \mathbb{R}^p$, let $r_{[1]}^2(\mathbf{b}) \leq \dots \leq r_{[n]}^2(\mathbf{b})$ be the ordered squared residuals $(y_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2$. The **least trimmed squares** (LTS) estimate minimizes

$$T(\mathbf{b}) = \sum_{k=1}^q r_{[k]}^2(\mathbf{b}),$$

the sum of the q smallest squared residuals. The **least median of squares** (LMS) estimate

minimizes $r_{\lfloor n/2 \rfloor}^2(\mathbf{b})$ instead.

With $q = \lfloor (n + p + 1)/2 \rfloor$ the breakdown point of LTS is close to $1/2$ (see Rousseeuw and Leroy 1987): the fit follows the best-fitting half of the data. Computing it is combinatorial, since $\min T$ is the minimum over q -subsets of their least squares criteria ([Exercise \(B1\)](#)). The practical algorithm rests on the **concentration step** of Rousseeuw and Van Driessen (2006).

Proposition 20.6.2. Concentration step

Let $\mathbf{b} \in \mathbb{R}^p$, let H be a set of q cases with the smallest squared residuals at $\mathbf{b}$, and suppose the rows of $\mathbf{X}$ in H have full column rank. Let $\mathbf{b}'$ be the least squares estimate computed from the cases in H . Then $T(\mathbf{b}') \leq T(\mathbf{b})$.

Proof. By the choice of H , $T(\mathbf{b}) = \sum_{i \in H} (y_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2$, and since $\mathbf{b}'$ minimizes the sum of squares over H , this is at least $\sum_{i \in H} (y_i - \mathbf{x}_{(i)}^\top \mathbf{b}')^2$. Any q squared residuals at $\mathbf{b}'$ sum to at least the q smallest, $T(\mathbf{b}')$. ■

Repeated concentration steps give a nonincreasing sequence of values of T , constant after finitely many steps since there are finitely many q -subsets. The algorithm starts from many random **elemental** fits through p cases and keeps the best result. It needs only one start whose p cases are clean: with a fraction ϵ of bad cases, N starts all fail with probability about $(1 - (1 - \epsilon)^p)^N$, and with $\epsilon = 0.2$, $p = 4$, $N = 9$ starts push this below 1% ([Exercise \(B2\)](#)). LTS with $q \approx n/2$ is inefficient; the MM-estimator of Yohai (1987) uses a high-breakdown fit only to start a redescending M-estimate with a fixed robust scale, keeping the high breakdown point and recovering efficiency. A simplified version, bisquare IRLS from LTS with the scale fixed at the median absolute LTS residual over 0.6745, is used below.

```
def lts(X, y, q, n_starts=500, seed=0):
    """Least trimmed squares: minimize the sum of the q smallest squared residuals.

    Random elemental starts (p cases), each followed by concentration steps: fit
    least
    squares to the current q cases, then keep the q cases with the smallest
    residuals."""
    rng = np.random.default_rng(seed)
    n, p = X.shape
    best, best_obj = None, np.inf
    for _ in range(n_starts):
        idx = rng.choice(n, size=p, replace=False)
        if np.linalg.matrix_rank(X[idx]) < p:
            continue
        beta = np.linalg.solve(X[idx], y[idx])
        for _ in range(100):
            # concentration steps
            keep = np.argsort((y - X @ beta) ** 2)[:q]
            new, *_ = np.linalg.lstsq(X[keep], y[keep], rcond=None)
            if np.allclose(new, beta):
                break
            beta = new
        obj = np.sort((y - X @ beta) ** 2)[:q].sum()
        if obj < best_obj:
```

```

        best, best_obj = beta, obj
    return best, best_obj

```

[Run this code online](#)

20.6.5 Using a robust fit diagnostically

The procedure: (1) fit least squares and a high-breakdown or MM-type fit; (2) compare the coefficients, since a difference of more than a standard error means a minority of cases drives least squares; (3) standardize the robust residuals by the robust scale and list cases beyond about 2.5 or with small weights, plotting them against a robust leverage measure to separate outlying responses from bad leverage points (Rousseeuw and van Zomeren 1990); (4) examine the flagged set substantively, refit without it, and report both fits. Theorem 20.5.1(d) measures its incompatibility with the rest, with an optimistic p -value because the set was chosen from the data.

Example. Robust fits on the masking data

For the data of Example (Four identical bad records) (Figure 20.6.1(a)), least squares has slope 0.007, and the Huber fit 0.005, with Huber weight 1.000 for each bad record: they have captured the fit so completely that their residuals are not even downweighted. The bisquare fit started from least squares is captured too (slope 0.005). LTS with $q = 18$ of the 34 cases has slope 0.546 and robust scale 0.587; in that scale each bad record has standardized residual -18.6 , and no clean case exceeds 2.81. The bisquare fit started from LTS has slope 0.509 and intercept 1.930, close to the clean fit (0.514 and 1.862), and gives the four records weight zero.

```

rng = np.random.default_rng(20)
n0, m = 30, 4
x_clean = np.sort(rng.uniform(0, 10, n0))
y_clean = 2 + 0.5 * x_clean + rng.normal(0, 0.5, n0)
x = np.r_[x_clean, np.full(m, 20.0)]
y = np.r_[y_clean, np.full(m, 2.0)]
X = np.column_stack([np.ones(len(x)), x])
n, p = X.shape

b_ls, *_ = np.linalg.lstsq(X, y, rcond=None)
b_hub, w_hub, _ = irls(X, y, huber_weight)
b_lts, _ = lts(X, y, q=(n + p + 1) // 2)
s_lts = mad_scale(y - X @ b_lts)
b_bis, w_bis, _ = irls(X, y, bisquare_weight, beta=b_lts, scale=s_lts)
for name, b in [("least squares", b_ls), ("Huber", b_hub), ("LTS", b_lts), ("
    bisquare from LTS", b_bis)]:
    print(f"{name:18s} intercept {b[0]:6.3f} slope {b[1]:6.3f}")
print("scaled LTS residuals of the cluster:", ((y - X @ b_lts) / s_lts)[-m:].round
    (1))

```

[Run this code online](#)

Example. Engel's budgets, robustly

For food expenditure on income in Engel's data (Example (Leverage in Engel's household budgets)), least squares gives slope 0.485, Huber 0.537 and bisquare 0.553. The bisquare fit

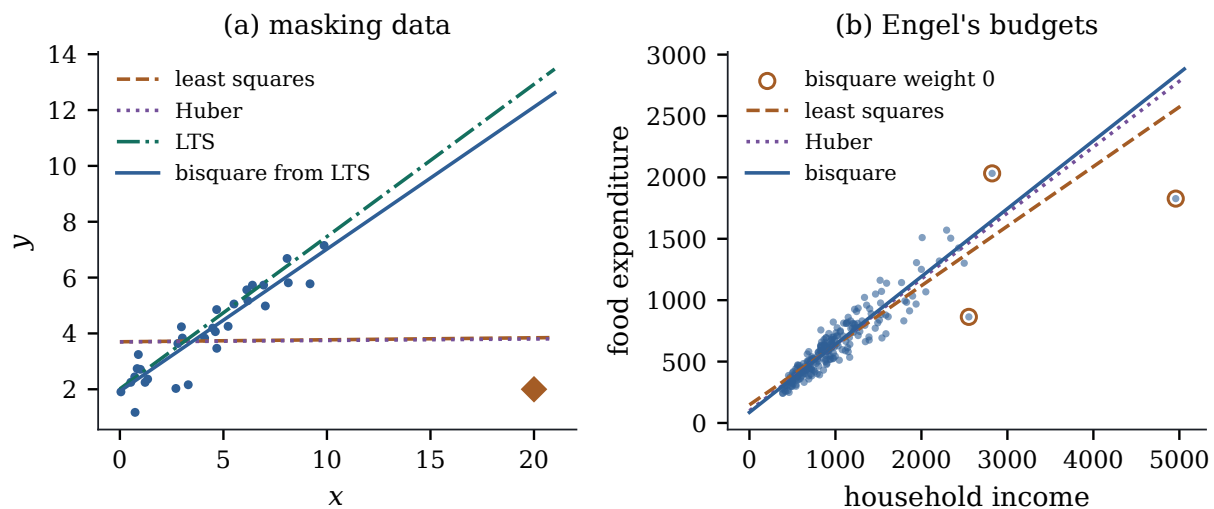


Figure 20.6.1: Robust fits. (a) The masking data: least squares and Huber are captured, LTS and bisquare from LTS are not. (b) Engel's budgets; circled points have bisquare weight zero.

gives weight zero to 3 households, rows 59, 105, 138 (Figure 20.6.1(b)). Row 138, the case of highest leverage (income 4958, food 1827, far below the line through the others, Huber weight 0.117), pulls the least squares slope down; in the least squares fit its externally studentized residual is -8.39 and its Cook's distance 9.28.

But the spread of food expenditure grows with income. On the logarithmic scale the three fits nearly agree (slopes 0.856, 0.868, 0.871) and the smallest bisquare weight is 0.16: nobody is rejected. The robust fit correctly reported that a few cases disagree with a constant-variance line; whether the data or the scale is wrong no fit can decide (Chapter 21, Chapter 22).

```
import statsmodels.api as sm
engel = sm.datasets.engel.load_pandas().data
inc = engel["income"].to_numpy()
food = engel["foodexp"].to_numpy()
XE = np.column_stack([np.ones(len(inc)), inc])
fits = {
    "least squares": sm.OLS(food, XE).fit(),
    "Huber": sm.RLM(food, XE, M=sm.robust.norms.HuberT()).fit(),
    "bisquare": sm.RLM(food, XE, M=sm.robust.norms.TukeyBiweight()).fit(),
}
for name, f in fits.items():
    print(f"{name:14s} slope {f.params[1]:.4f}")
w = fits["bisquare"].weights
print("households with bisquare weight 0 (row numbers from 1):", np.where(w == 0)[0]
      + 1)
```

[Run this code online](#)

Warning. Deleting flagged cases and then reporting ordinary standard errors treats a data-driven choice as fixed in advance, and the intervals are too short. Report both analyses; Chapter 23 discusses resampling that can include the selection step.

20.6.6 Exercises

20.6.6.1 A. Check your understanding

Exercise. A1

With $k = 1.345$, compute the Huber weights of standardized residuals 0.5, 2 and 10. With $c = 4.685$, compute the bisquare weights of the same residuals.

Exercise. A2

Show that least squares is the M-estimator with $\rho(u) = u^2/2$, that its weights are all one, and that IRLS started anywhere reaches it in one step.

20.6.6.2 B. Practice

Exercise. B1

Show that $\min_{\mathbf{b}} T(\mathbf{b}) = \min_H \text{SSE}_H$, where H ranges over the q -subsets of cases for which the rows of $\mathbf{X}$ in H have full column rank and SSE_H is the residual sum of squares of least squares on H (assume at least one such subset exists). Deduce that an LTS estimate is the least squares estimate on some q -subset.

Solution. For any $\mathbf{b}$ and any q -subset H , $T(\mathbf{b}) \leq \sum_{i \in H} (y_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2$, with equality when H consists of cases with the q smallest squared residuals. So $T(\mathbf{b}) = \min_H \sum_{i \in H} (y_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2$, and $\min_{\mathbf{b}} T(\mathbf{b}) = \min_H \min_{\mathbf{b}} \sum_{i \in H} (y_i - \mathbf{x}_{(i)}^\top \mathbf{b})^2$, the order of minimization being interchangeable. For a subset of full rank the inner minimum is SSE_H , attained at the least squares estimate on H . (Subsets without full rank can be handled by a limiting argument or excluded, as the statement assumes.) An LTS estimate is therefore the least squares estimate on a minimizing H .

Exercise. B2

Find the smallest number N of elemental starts for which $(1 - (1 - \epsilon)^p)^N \leq 0.01$ when $\epsilon = 0.2$ and $p = 4$, and when $\epsilon = 0.4$ and $p = 10$. What does the second answer say about high-breakdown fitting with many regressors?

Exercise. B3

For the state murder-rate regression with all 51 jurisdictions, fit Huber and bisquare M-estimates with statsmodels and list the jurisdictions with the smallest weights. Compare the poverty coefficient with the least squares values with and without the District of Columbia (Example (How much does the District of Columbia matter?)).

20.6.6.3 C. Going deeper

Exercise. C1

Consider regression through the origin, $E(Y) = \beta x$, and an M-estimate with fixed scale $\hat{\sigma} = 1$ and a continuous, nondecreasing, bounded, odd ψ with $\psi(u) > 0$ for $u > 0$. Add to fixed data (x_i, y_i) , $i < n$, one case $(x_n, y_n) = (L, aL)$. Show that as $L \rightarrow \infty$ every solution $\hat{\beta}_L$ of $\sum_i \psi(y_i - \hat{\beta}x_i)x_i = 0$ tends to a . Conclude that the breakdown point is $1/n$.

Solution. Let $B = \sup|\psi|$ and $S = \sum_{i < n} |x_i|$, so the first $n-1$ terms contribute at most BS in absolute value, whatever $\hat{\beta}$. Fix $\eta > 0$. If $\hat{\beta} \leq a - \eta$, the last term is $\psi(L(a - \hat{\beta}))L \geq \psi(L\eta)L$, which exceeds BS once L is large, because $\psi(L\eta) \rightarrow \sup \psi > 0$ (as ψ is nondecreasing and positive on $(0, \infty)$). Then the sum is positive and $\hat{\beta}$ is not a solution. Symmetrically for $\hat{\beta} \geq a + \eta$. So for large L every solution lies in $(a - \eta, a + \eta)$. Since a is arbitrary, one case can place the estimate anywhere, and the breakdown point is $1/n$.